Hyperdimensional decoding of spiking neural networks

NEUROMORPHIC
Computing and Engineering

OPEN ACCESS

RECEIVED
12 November 2025

REVISED
25 February 2026

ACCEPTED FOR PUBLICATION
9 March 2026

PUBLISHED
19 March 2026

PAPER

Hyperdimensional decoding of spiking neural networks

Cedrick Kinavuidi*, Luca Peres and Oliver Rhodes

International Centre for Neuromorphic Systems, University of Manchester, Manchester, United Kingdom
* Author to whom any correspondence should be addressed.

E-mail: cedrick.kinavuidi@manchester.ac.uk, luca.peres-2@manchester.ac.uk and oliver.rhodes@manchester.ac.uk

Keywords: spiking neural networks, hyperdimensional computing, SNN decoding, event-based processing, neuromorphic algorithms

Abstract

This work presents a novel spiking neural network (SNN) decoding method, combining SNNs with hyperdimensional computing (HDC). This decoding method is designed to achieve high accuracy, high noise robustness, low inference latency and low energy consumption. Compared to analogous architectures decoded with existing approaches, the SNN-HDC model attains generally better classification accuracy, lower inference latency, lower spike count and lower estimated energy consumption on multiple test cases from the literature. The SNN-HDC achieved spike count reductions of 1.74 × to 3.36 × on the DvsGesture dataset and 1.36 × to 2.70 × on the SL-Animals-DVS dataset. The SNN-HDC achieved estimated energy consumption reductions of 1.24 × to 3.67 × on the DvsGesture dataset and 1.38 × to 2.27 × on the SL-Animals-DVS dataset. The proposed decoding method enables detection of classes unseen during training. On the DvsGesture dataset, the SNN-HDC model can detect 100% of samples from an unseen/untrained class. The findings suggest the proposed decoding method is a compelling alternative to both rate and latency decoding.

1. Introduction

Spiking neural networks (SNNs) are the third generation of neural network models [1], conceptualised following artificial neural networks (ANNs), but inspired by spiking neurons in the brain. Although SNNs have the potential to replace ANNs, partly due to the prospect for significant energy savings [2], SNNs are less commonly employed. SNNs tend to achieve lower accuracy than ANNs in several learning tasks [3]. The energy efficiency of SNNs is only realised on specialised neuromorphic hardware [4]. Implementing SNNs is more complex than ANNs, as it requires knowledge of both neuroscience and machine learning [5]. Neuromorphic algorithms are critical to the maturation and wider spread adoption of SNNs. Although SNNs are designed to be more biologically plausible than ANNs, several methods commonly used by SNNs deviate from known biological mechanisms. Examples include training using backpropagation through time (BPTT) and disregarding the temporal aspect of spikes. This paper focuses on improving concept representations and SNN output decoding by developing SNN specific methodology.

By combining SNNs and hyperdimensional computing (HDC), we propose a model that builds on the hypothesis the brain uses distributed representations. This paper expands the limited body of existing research on the combination of SNNs and HDC. The work presented focuses on how modifications to the SNN output layer and decoding method impact performance. We contribute an SNN that directly outputs hypervectors which are decoded using HDC. The effectiveness of this model and decoding technique is evaluated against comparable rate and latency decoded models. Performance is evaluated using metrics including classification accuracy, inference latency and relative energy consumption. Characteristics exclusive to the SNN-HDC model, such as the ability to detect classes unseen during training, are also analysed.

The paper is structured into the following sections. Section 2 provides an overview of fundamental concepts. Section 3 discusses past research related to this work, including literature on combining SNNs with HDC. Section 4 describes the proposed methodology, detailing: how SNNs are combined with

HDC to create a new architecture, how this architecture transforms data into hypervectors, how it is trained and how experiments were performed. Section 5 presents the experimental results. Section 6 summarises the paper and draws conclusions.

2. Background

2.1. SNNs

SNNs are designed to be more biologically plausible than ANNs in order to leverage more of the brains characteristics, primarily its energy efficiency. Mimicking the brain, SNNs propagate information using binary signals with an inherent temporal aspect, referred to as spikes. These spikes are typically sparse in time, and processed in an event-driven manner across the entire network. In contrast, ANNs typically propagate dense vectors of numerical neuronal activations and process all neuronal activations at synchronous fixed intervals. This difference is a key factor in why SNNs have the potential for superior energy efficiency compared to ANNs [2]. A notable property of the brain is that its energy consumption scales proportionally to the number of spikes processed [6]. This property can be achieved by combining neuromorphic hardware with an SNN [7]. Neuromorphic hardware is a specialised type of hardware inspired by the dynamics observed in the brain and designed to emulate the brain's characteristics, particularly asynchronous event-driven processing [8].

Although SNNs can be trained to use a low number of spikes [9], the most widely used approach, rate decoding [10], relies on high levels of spiking activity to perform well [11]. Rate decoding involves distinguishing between the number of spikes fired by the output neurons over a specified temporal window. This necessitates sufficient internal spiking activity such that the SNN can output enough spikes to make a clear distinction between outputs. While this method typically produces the highest accuracy SNNs, the models developed exhibit high inference latency [10], as calculating rates effectively requires accumulating spikes over time. The resulting spiking activity also leads to high energy consumption, in some cases requiring more energy than an equivalent ANN [12]. This issue stems from translating methods developed for ANNs by making value intensity proportional to the number of spikes, rather than developing methods specific to SNNs accounting for their inherent temporal properties.

Insights from neuroscience research can inform new neuromorphic approaches. The neural and synaptic updates that occur in the brain are primarily based on local information [13], allowing multiple neurons to respond to the same external stimulus. Approximately 85 000 neurons die in the healthy adult brain every day [14]. However, this neuronal death does not result in continuous loss of learned concepts. These observations suggest that the brain represents concepts in a distributed fashion across a population of neurons. These distributed representations likely exist in a way that enables the brain to achieve low inference latency [15], low energy consumption [16], and high robustness [14].

2.2. HDC

HDC is a brain-inspired computational model that abstracts the physical structure of the brain and focuses on how the brain can represent and compare concepts with populations of neurons [17]. HDC performs computations on hypervectors, which are high dimensional vectors comprising the output or state of a group of neurons, mimicking the distributed representations in the brain. The large number of dimensions, as well as their holographic representation, make hypervectors robust to noise. Figure 1 shows two binary hypervectors each with a dimensionality of 10. The alignment of two hypervectors provides a measure of their similarity, with high alignment indicating conceptual similarity and low alignment indicating conceptual dissimilarity.

Cosine similarity (equation (1)) is a commonly used method for comparing two hypervectors. Cosine similarity values lie within the range $-1 \leqslant x \leqslant 1$. -1 represents complete dissimilarity, 0 represents orthogonality and 1 represents complete overlap. The cosine similarity of the binary hypervectors A and B from figure 1 is 0.447. As calculating cosine similarity involves a dot product, dimensions containing zeros do not contribute to the similarity. Calculating more informative similarity values requires changing the zeros to -1. Accordingly, the cosine similarity between hypervectors A and B is 0,

$$\cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}. \tag{1}$$

Binary hypervectors can also be compared through a computationally efficient metric called Hamming distance which counts the number of mismatches across corresponding dimensions (equation (2)). Normalised Hamming distance is calculated by dividing the Hamming distance by the number of dimensions (equation (3)). Normalised Hamming distance values lie within the range

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$
$$B = \begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 1. Two random binary hypervectors, A and B, each with 10 dimensions.

$0 \leqslant x \leqslant 1$. 0 represents complete overlap, 0.5 represents orthogonality and 1 represents complete dissimilarity. Equation (2) shows how the Hamming distance is calculated between binary hypervectors A and B from figure 1, resulting in a value of 5. As hypervectors A and B have a dimensionality of 10, the normalised Hamming distance is 0.5. Hamming distance can be considered more suitable for neuromorphic systems where a goal is to minimise energy consumption,

$$\mathrm{HD} = \sum_{i=1}^{N} A_i \oplus B_i \tag{2}$$

$$\mathrm{HD}_{\mathrm{norm}} = \frac{1}{N} \sum_{i=1}^{N} A_i \oplus B_i. \tag{3}$$

2.3. SNN decoding methods

SNN decoding is the process of interpreting the outputs of an SNN to produce insights. SNNs output spikes over time, allowing both the number and timing of spikes to be used for decoding. Neural networks used for classification, trained with supervised learning, typically target representations that are one-hot decoded. The output layer is composed of one neuron per target class. The target activity in each output neuron is dependent on the decoding method.

Rate decoding classifies inputs based on the output neuron with the highest spike count. This is the most commonly used decoding method in SNN research as it typically achieves the highest classification accuracies. The disadvantages of this method are relatively high energy consumption [12] and high inference latency [10]. Equation (4) defines how the mean squared error (MSE) of rate decoding is calculated. N denotes the number of samples, r the output normalised firing rates and $\hat{r}$ the target normalised firing rates. It is generally accepted that rate decoding is not the dominant coding method in the brain, as it does not address the inference latency observed in the brain [18],

$$\mathrm{MSE} = \frac{1}{N} \sum_{i=1}^{N} (r_i - \hat{r}_i)^2. \tag{4}$$

Latency decoding classifies inputs based on the output neuron that fires a spike first. This method typically results in more energy efficient and lower inference latency SNNs compared to rate decoding [19], due to processing fewer spikes and output neurons being trained to fire earlier. However, as it typically achieves lower classification accuracies [11] and is less noise robust [10], it is not as commonly employed. Equation (5) defines how the MSE of latency decoding is calculated. N denotes the number of samples, l the output normalised latencies of the first spike from each neuron and $\hat{l}$ the target normalised latencies of the first spike from each neuron. It is generally accepted that latency decoding is not the dominant coding method in the brain, as it does not address how information is accumulated over time in the brain to make a classification [20],

$$\mathrm{MSE} = \frac{1}{N} \sum_{i=1}^{N} \left(l_i - \hat{l}_i\right)^2. \tag{5}$$

Population decoding applies a decoding method using multiple neurons per classification rather than a single neuron. This mitigates the issues associated each decoding method. When population decoding is combined with rate decoding, inference latency is reduced because output rate is interpreted across multiple output neurons. However, energy consumption increases due to more neurons firing. When population decoding is combined with latency decoding, noise robustness improves as a single erroneous

spike does not solely determine the network output. However, inference latency typically increases as more spikes are required to produce a confident output.

While these methods have demonstrated effectiveness on samples of data [21, 22], they have not addressed inference with continuous data in an event-driven manner without introducing additional caveats including dynamic network resetting [23] and adaptive cut-offs [24]. Rate decoding requires counting spikes, raising the question of when counting should start for a classification. Latency decoding relies on the first spike output, raising the question of what classifies as 'first' in a system that is always outputting data. Although population decoding addresses the implausibility of single neuron representations, it does not answer how an SNN could harness distributed representations with high accuracy, high noise robustness, low inference latency and low energy consumption.

2.4. Representation expressiveness

This subsection compares the representation expressiveness of one-hot decoding and the theoretical representation expressiveness of binary hypervectors. Representation expressiveness is defined as the number of classes that a decoding method can represent given a particular vector size (number of dimensions). This expressivity impacts both computation and memory resources, with decoding methods with higher expressiveness able to represent larger numbers of classes with fewer computational resources. This can lead to improved efficiency when representing systems with large numbers of output classes in resource-constrained environments.

One-hot decoding is the prevalent method used in neural networks for representing multi-class categorical data [25]. One-hot decoding involves representing the output layer of a neural network as a vector with one dimension per class. Classifications are typically performed by selecting the dimension with the largest value. For SNNs, the largest value corresponds to either the highest number of spikes fired or the earliest spike. As one-hot decoding can only express one class per dimension, its representation expressiveness scales linearly.

The representation expressiveness of binary hypervectors scales exponentially rather than linearly. Consider a binary hypervector with an equal probability of every dimension being either 0 or 1, as shown in equation (6). For a hypervector of size D, we define the expressiveness as the number of hypervectors N that can be generated such that every pair of hypervectors is pseudo-orthogonal. Orthogonal hypervectors have a normalised Hamming distance of 0.5. Here we consider pseudo-orthogonal hypervectors as having a normalised Hamming distance within 0.5 ± 0.05. The statistical procedure to estimate this value is shown below,

$$\begin{gathered} \mathbf{H} = [H_1, H_2, \ldots, H_D] \\ \text{for } i = 1, \ldots, D \\ P(H_i = 1) = 0.5 \\ P(H_i = 0) = 0.5. \end{gathered} \tag{6}$$

A single dimension in a binary hypervector follows a Bernoulli distribution with equal probability $p = 0.5$ for either outcome. The variance of a single dimension is defined in equation (7). This variance is then used to calculate the variance of an entire binary hypervector with dimensionality D, from which the standard deviation is derived,

$$\begin{gathered} \mathrm{Var}(X) = p(1-p) \\ \mathrm{Var}(D_i) = 0.5(1-0.5) \\ \mathrm{Var}(D_i) = 0.25 \\ \mathrm{Var}(D) = D/4 \end{gathered} \tag{7}$$

$$\mathrm{SD} = \sqrt{\mathrm{Var}(X)}$$

$$\mathrm{SD} = \frac{\sqrt{D}}{2}.$$

The pseudo-orthogonal bounds defined previously are normalised Hamming distances in the range $0.45D \leqslant x \leqslant 0.55D$. These bounds, with a mean of $0.5D$ and the standard deviation defined

in equation (7), are used to calculate the Z-scores of the pseudo-orthogonality bounds as shown in equation (8),

$$Z_{\text{score}} = \frac{\text{Raw Score} - \text{Mean}}{\text{Standard Deviation}}$$

$$\begin{aligned} Z_{\text{lower}} &= \frac{0.45D - 0.5D}{\sqrt{D}/2} \\ Z_{\text{lower}} &= -0.1\sqrt{D} \end{aligned} \tag{8}$$

$$\begin{aligned} Z_{\text{upper}} &= \frac{0.55D - 0.5D}{\sqrt{D}/2} \\ Z_{\text{upper}} &= 0.1\sqrt{D}. \end{aligned}$$

The Hamming distance between two randomly generated hypervectors follows a normal distribution [26]. The cumulative distribution function is used to calculate the probability that two randomly generated hypervectors fall within the specified pseudo-orthogonality bounds, shown in equation (9). Let P denote the probability that a pair of hypervectors is pseudo-orthogonal and $(1-P)$ denote the probability of non-pseudo-orthogonality. The total number of pairs of N hypervectors is expressed as $\frac{N(N-1)}{2}$. These two expressions are combined to calculate the number of binary hypervectors that can be generated randomly such that at most one pair is outside of the previously defined Hamming distance bounds. The expression is rearranged to a quadratic equation solved for N. This value is the upper bound on the number of classes that a hypervector of size D can represent,

$$\begin{aligned} P &= \text{CDF}\left(Z_{\text{upper}}\right) - \text{CDF}\left(Z_{\text{lower}}\right) \\ \frac{N(N-1)}{2}(1-P) &= 1 \\ (1-P)N^2 - (1-P)N - 2 &= 0. \end{aligned} \tag{9}$$

Figure 2 plots the expressiveness of both one-hot decoding and binary hypervectors showing that for small dimensionalities the expressiveness of binary hypervectors is lower than that of one-hot decoding, however, for large dimensionalities the opposite is true. At $D = 2633$ the expressiveness of both is approximately the same. Beyond this value the expressiveness of hypervectors increases rapidly and quickly exceeds practical expressiveness requirements. DeepSeek-V3 (671B) has a vocabulary size of 129 280 [27], corresponding to an equal number of dimensions when represented with one-hot decoding. A binary hypervector with the same number of dimensions can represent approximately 2.2×10^{141} classes. To represent 129,280 classes, a binary hypervector would require 4148 dimensions.

The theoretical expressiveness scaling of binary hypervectors has significant potential benefits to future models using large numbers of classes. However, to our knowledge there is no publicly available neuromorphic dataset that is large enough to test this scaling property. The work presented in this paper explores smaller problems as a way to better understand the interaction between SNNs, HDC, spatio-temporal event-based data and spiking activity. It is hoped that the understanding gained from this paper will motivate further scaling experiments and the production of future datasets able to exploit these techniques.

2.5. Neuromorphic vision

Neuromorphic vision data is employed in place of conventional frame based data as its temporal binary events are well suited for SNN processing [2]. Real-world neuromorphic vision data is captured using a neuromorphic sensor, such as a dynamic vision sensor (DVS) [28] which mimics the neurobiological structures and functionalities of the retina [29]. While a conventional frame-based vision sensor outputs frames (images) which contain data for every pixel, a DVS only outputs events. An event is a threshold change in luminescence that occurs at a specific time to a specific pixel. An event stream is a series of events over time. Conceptually, a conventional sensor 'sees' everything, while a DVS only 'sees' movement. Neuromorphic sensors typically offer higher energy efficiency and temporal resolution compared to conventional frame-based sensors [29]. The DAVIS event-based camera [30] has a resolution of 240×180, a latency of $3\,\mu$s and a power consumption of 5–14 mW. A comparable conventional high speed frame-based camera [31] has a resolution of 256×256, a latency of $429\,\mu$s and a power consumption of 4 W.

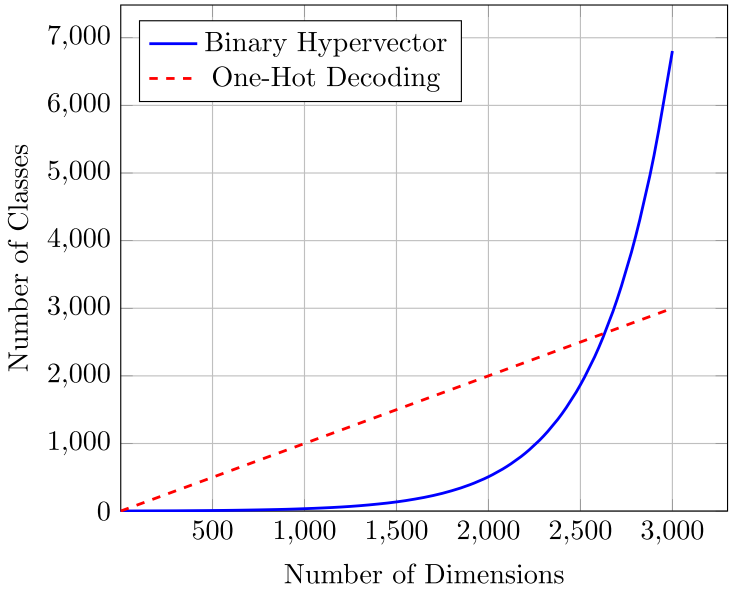


Figure 2. Number of classes that binary hypervectors and one-hot decoding can represent given a number of dimensions.

2.6. Temporal resolution of neuromorphic data

SNN models in recent literature temporally compress multiple seconds of neuromorphic data into a small number of timesteps such as 16 [32], 20 [33, 34] or 25 [35]. This is typically done to reduce training time. While this method is commonly used and the SNNs that use it achieve high accuracies, there are multiple shortcomings with this approach. Firstly, the input frames can cover differing timescales. For example, in the neuromorphic vision dataset DvsGesture [36], the shortest sample in the dataset is 1.7 s while the longest is 18.5 s. If both of these samples are compressed into 25 frames, then each frame in the longer sample contains a larger temporal window than each frame in the shorter sample. Given that SNNs make use of temporal information [19], this raises the question of the appropriateness of inputting differing timescales as well as what timescale one should use for inference of a data sample with an unknown length. Secondly, as data is being compressed temporally, there is a risk that temporal information is being diminished. Thirdly, in real-world deployment scenarios, compressing frames has a negative impact on the inference latency of the model. Due to causal constraints the complete processing of a sample cannot precede the sample's acquisition. Consider a model that is trained on data samples which are 10 s long compressed into 25 equal frames. Should this model be deployed in the real world it could only produce an output once every 400 ms. Compare this to a model that is trained with consistent 1 ms frames. This model could produce an output once every millisecond. Deployed in the real world both of these models would take the same amount of time to fully process each data sample, however, since the latter model has a higher temporal resolution, it has the potential for lower inference latency. It is desirable to move towards solutions that are tailored towards real-world performance rather than focusing purely on improving dataset accuracy.

3. State-of-the-art

While there is relatively limited research on the topic of combining SNNs and HDC, the little research that does exist is promising.

SpikeHD [37] developed a convolutional SNN adapted for inference with HDC. The authors trained a convolutional SNN using deep continuous local learning (DECOLLE) [38] using smooth L1 loss. The SNN was trained on neuromorphic data including the DvsGesture dataset [36] and MNIST dataset [39] (Poisson generated spikes). Following training, the last layer of the SNN was removed, converting it into a feature extractor. After data samples were passed through this feature extractor, the outputs were transformed into hypervectors using projection matrices. These hypervectors were then used to train an HDC model. The authors compared the performance of the full SNN and the feature extractor combined with the HDC. The combined model achieved 5.7% and 3.2% higher classification accuracy than the pure SNN on MNIST [39] and DvsGesture [36], respectively. The authors attribute this performance improvement to the two-stage information processing.

HyperSpike [40] introduced a single-layer, untrained SNN combined with HDC, achieving performance comparable to surrogate gradient trained SNNs. After a data sample was passed through the untrained SNN, the outputs were transformed into hypervectors using projection matrices. These hypervectors were used to train the HDC model. This combined model was then compared to SNNs trained on a range of neuromorphic vision datasets. The combined model achieved 0.2% higher classification accuracy on the N-MNIST dataset [41], 6.6% lower classification accuracy on the DvsGesture dataset [36] and 1.5% lower classification accuracy on the ASL-DVS dataset [42] relative to the surrogate gradient trained SNNs. These results indicate that HDC can effectively extract information from an SNN, despite the lack of SNN training.

The research described above shows promising results, however, there are potential improvements. SpikeHD did not initially train its SNN directly on hypervector representations of each class. This resulted in the spiking activity of the SNN being optimised to target a different representation of the classes. The combined model was later updated by what is effectively transfer learning. This approach increases training time and computational cost. Furthermore, transfer learning can result in worse performance than models that are trained from scratch [43]. A combined model trained to target hypervectors initially may achieve better performance. Although HyperSpike did not train the SNN and achieved competitive performance, training the SNN portion would result in better performance [44] by increasing the effectiveness of the feature extractor.

Both SpikeHD and HyperSpike require matrix multiplications to transform the SNN outputs into hypervectors. Although matrix multiplication has been optimised on GPUs, avoiding the reintroduction of matrix multiplication to SNNs helps to maintain the performance benefits of neuromorphic hardware. One way to make this hypervector generation process more energy efficient, which could also be considered more neuromorphic, would be removing multiplication and relying solely on addition, as multiplication requires considerably more energy than addition [45]. A 45 nm processor consumes 0.1 pJ for a 32-bit addition and 3.1 pJ for a 32-bit multiplication [45].

Both approaches process the entire data sample through the SNN before transforming the SNN output into a hypervector, negatively impacting the end to end inference latency. Predictions cannot be made before the full temporal length of a sample has been observed. In contrast, both rate and latency decoding allow confident outputs to be made during a sample input. A hypervector generation process applied in a continuous event-driven manner would reduce inference latency and enable continuous predictions.

4. Methodology

The differences between a standard rate decoded SNN, latency decoded SNN and the proposed model in this paper are shown in figure 3. Both the rate decoded SNN (figure 3(a)) and the latency decoded SNN (figure 3(b)) have one output neuron per class (one-hot decoding). The proposed model, termed SNN-HDC (figure 3(c)), has an experimentally determined number of output neurons. The proposed hypervector generation process uses the presence and absence of output spikes as binary dimensional values. Each output neuron corresponds to its own dimension in the hypervector and each dimension is initialised to zero. A neuromorphic data sample captured with a DVS [28] is processed by the SNN. Simultaneously with the data input, the presence of spikes from an output neuron causes the corresponding dimension to be flipped from zero to one. This hypervector is then compared with known hypervectors using Hamming distance (equation (2)) to produce classifications.

The class hypervectors are generated prior to training according to equation (6). The class hypervectors are all binary and are generated with each dimension having a 50% chance to be either 0 or 1. Although this generation is random, due to the high number of dimensions that hypervectors possess, it results in hypervectors that are all pseudo-orthogonal to each other [17]. Binary hypervectors were chosen over non-binary hypervectors as they allow for more energy efficient comparisons as discussed in section 2.2. This method presumes that output neurons react to specific temporal features and that the frequency and timing of their spiking patterns is not as important as their presence.

4.1. Neuromorphic data

This paper utilises multiple neuromorphic datasets to explore how the method responds to different types of data. All data in this work utilises a one millisecond temporal resolution. This ensures temporal consistency of inputs, reduces the risk of diminishing temporal information and increases the output temporal resolution of each model.

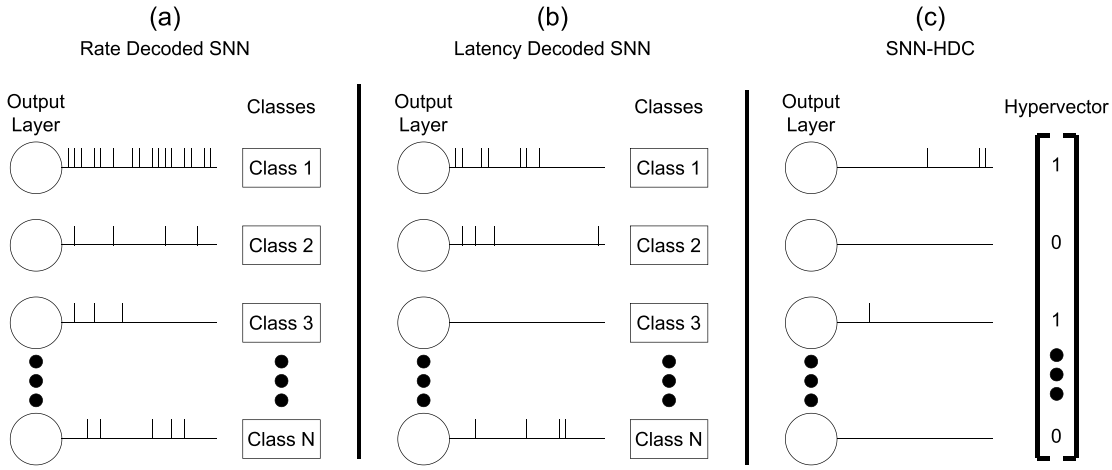


Figure 3. A rate decoded SNN (A), a latency decoded SNN (B) and the SNN-HDC model (C). The rate and latency decoded SNNs have one output neuron per classification. Rate decoded outputs are classified based on highest spike count. Here the rate decoded SNN (A) would decode the output as Class 1. Latency decoded outputs are classified based on the earliest fired spike. Here the latency decoded SNN (B) would decode the output as Class 2. The SNN-HDC (C) has an experimentally determined number of output neurons used to build a hypervector over time. Every dimension initialises with a value of 0. This value is changed to 1 if that dimension observes the presence of spikes.

4.1.1. DvsGesture

The DvsGesture dataset [36] contains eleven classes of various hand gestures, with a total of 1341 data samples. The Tonic [46] library is used to load the data and provides the predefined train/test split of 1077/264, with 24 samples of each class composing the test set.

The data is spatially downsampled to 32×32, as presented in the original DvsGesture dataset [36] paper, to help efficient model exploration. This spatial downsampling also matches that used in referenced works such as SQUAT [35], which inspired the architecture employed here and allows for direct comparisons of results. Model architectures are further elaborated in section 4.2. The data is also temporally truncated, with only the first 1500 ms of each data sample used. This enables consistent sample lengths across the dataset, avoiding the discrepancies in sample length between classes/individuals (e.g. the shortest sample length is 1.7 s and the longest 18.5 s). This approach follows that presented by the SLAYER paper [47], enabling direct comparison of results.

4.1.2. SL-Animals-DVS

The SL-Animals-DVS dataset [48] contains nineteen classes of various sign language words of animals being performed. A total of 59 people performed each of the 19 signs under four different lighting conditions. The dataset paper [48] contains results for models trained on all four lighting conditions as well as separate models trained on a subset of three lighting conditions. This was due to one of the lighting conditions introducing significant noise to the data samples making classification harder [48]. The results presented in this paper use all four lighting conditions to evaluate performance on all real world conditions, including noisy samples. As there is no predefined train/test split, a strategy of K-fold cross validation with a leave-signers-out approach is used, following the methodology used by the dataset paper [48].

The data is spatially downsampled to 32×32, as presented in the original SL-Animals-DVS dataset [48] paper, to help efficient model exploration. This spatial downsampling also matches the SL-Animals-DVS [48] model that inspired the architecture used here, allowing for direct comparison of results. Model architectures are further elaborated in section 4.2. Each sample is temporally truncated to the first 1500 ms, as in the original dataset paper [48]. This time frame allows every sign to be performed while avoiding overlaps [48], enabling direct comparison of results.

4.2. SNN architecture

The models defined in this work use a relatively small number of layers and parameters compared to SNN models that currently achieve the highest accuracies on both the DvsGesture dataset [21, 33, 34] and the SL-Animals-DVS dataset [49, 50]. The objective of this paper is not to develop large models to attain state-of-the-art accuracy but instead to compare the effectiveness of different SNN decoding techniques. Using a small model already validated by related literature [35, 48] allows for faster experimentation while facilitating thorough analysis of SNN decoding techniques.

Table 1. Architectures used for the DvsGesture dataset [36]. *D* refers to a fully connected layer with *D* number of LIF neurons. SNN-1 and SNN-2 are trained with rate and latency decoding giving 5 networks in total: SNN-HDC, Rate-1, Rate-2, Latency-1 and Latency-2.

SNN-HDC	SNN-1 (Same depth)	SNN-2 (Deeper)
16Conv5	16Conv5	16Conv5
BatchNorm	BatchNorm	BatchNorm
Maxpool	Maxpool	Maxpool
Dropout	Dropout	Dropout
32Conv5	32Conv5	32Conv5
BatchNorm	BatchNorm	BatchNorm
Maxpool	Maxpool	Maxpool
Dropout	Dropout	Dropout
D	11	*D*
		11

Different architectures will be used for each of the neuromorphic datasets. This shows that the proposed decoding method works consistently regardless of the architecture used. All the models used in this paper are convolutional SNNs with leaky-integrate-and-fire (LIF) neurons.

The SNN-HDC method proposed in this work entails both algorithmic changes for loss calculation and inference as well as architectural changes, specifically, increasing the number of the neurons in the output layer, relative to a standard rate/latency decoded SNN. Due to these changes in architecture, the SNN-HDC is compared with multiple analogous SNNs for fair analysis. Three types of model are tested for each dataset: the SNN-HDC; a model of the same depth (SNN-1); and a model that is one layer deeper than the SNN-HDC (SNN-2). The architectural differences between the three model types are summarised in tables 1 and 2 and described in detail below.

The architectural difference between the SNN-HDC and the same depth model is in the output layer. The SNN-HDC model has an experimentally determined number of output neurons, whereas the same-depth model is one-hot decoded. The deeper model copies the architecture of the SNN-HDC, and appends a one-hot decoded output layer. Comparing the SNN-HDC to an architecture of the same depth isolates the effect of replacing an SNN output layer with a hyperdimensional layer. Comparing the SNN-HDC to a deeper architecture assesses whether the performance differences are due to increasing the number of neurons in the output layer. The same depth (SNN-1) and deeper (SNN-2) architectures are trained using both rate and latency decoding resulting in four SNNs referred to as Rate-1, Rate-2, Latency-1 and Latency-2.

The DvsGesture SNN-HDC uses an architecture of [16*c*5–*bn*–2*p*–0.2*d*–32*c*5–*bn*–2*p*–0.2*d*–*D*]. Here *c* denotes a convolutional layer, *bn* denotes batch normalisation, *p* denotes a maxpool operation, *d* denotes a dropout layer and *D* denotes a fully connected output layer with *D* number of neurons. This architecture is based on recently presented work on the same dataset [35]. The Rate-1 and Latency-1 networks replace *D* with 11 neurons. The Rate-2 and Latency-2 networks append 11 neurons after *D*. The 11 neurons form the one-hot decoded output layer. The model architectures are summarised in table 1. The performance of the baseline models, the proposed SNN-HDC model and previously published models are shown in table 7.

The SL-Animals-DVS SNN-HDC uses an architecture of [8*c*5–2*p*–16*c*5–2*p*–32*c*5–2*p*–25*fc*–*D*]. Here *c* denotes a convolutional layer, *p* denotes a maxpool operation, *fc* denotes a fully connected layer and *D* denotes a fully connected output layer with *D* number of neurons. This architecture is based on the architecture presented in the original dataset paper [48] albeit with smaller and fewer convolutional kernels due to available resources. The Rate-1 and Latency-1 networks replace *D* with 19 neurons. The Rate-2 and Latency-2 networks append 19 neurons after *D*. The 19 neurons form the one-hot decoded output layer. The model architectures are summarised in table 2. The performance of the baseline models, the proposed SNN-HDC model and previously published models are shown in table 9.

The LIF neurons used by all models in this work are defined in equations (10) and (11). Equation (10) shows the synaptic integration and leakage of a LIF neuron. The membrane voltage at the current timestep t is defined as $M_i[t]$, the decay rate is defined as β, the synaptic weight matrix is defined as W_{ij}, bias values defined as I_i and the presence or absence of input spikes at the current timestep is defined as $X_j[t]$. The value of β is determined experimentally for each model. Equation (11) shows that a LIF neuron fires a spike and resets its membrane to 0 if the membrane reaches the threshold value of 1. The snnTorch library [19] is used for training. The LIF neurons make use of the library's reset delay functionality which makes an LIF neuron propagate spikes one timestep after the membrane voltage crosses threshold. This is used to prevent spikes passing through multiple layers of

Table 2. Architectures used for the SL-Animals-DVS dataset [48]. *D* refers to a fully connected layer with *D* number of LIF neurons. SNN-1 and SNN-2 are trained with rate and latency decoding giving 5 networks in total: SNN-HDC, Rate-1, Rate-2, Latency-1 and Latency-2.

SNN-HDC	SNN-1 (Same depth)	SNN-2 (Deeper)
8Conv5	8Conv5	8Conv5
Maxpool	Maxpool	Maxpool
16Conv5	16Conv5	16Conv5
Maxpool	Maxpool	Maxpool
32Conv5	32Conv5	32Conv5
Maxpool	Maxpool	Maxpool
25	25	25
D	19	*D*
		19

the SNNs in a single timestep. This is advantageous when designing implementations for neuromorphic hardware, which typically requires at least one timestep to route spikes between neurons [51],

$$M_i[t] = \beta M_i[t-1] + W_{ij}X_j[t] + I_i \tag{10}$$

$$M_i[t], X_i[t] = \begin{cases} (0,1), & \text{if } M_i[t] \geqslant 1, \\ (M_i[t], 0), & \text{otherwise.} \end{cases} \tag{11}$$

4.3. Model training

The snnTorch library [19] is used to train the SNN models, with all models trained using surrogate gradients and BPTT. All models are trained on the datasets described in section 4.1, with a batch size of 32 and the default Adam optimiser parameters [52]. All models are trained across three different runs using three different PyTorch [53] seeds affecting the weight initialisation of all models and the generated class hypervectors for the SNN-HDC models. The primary difference between the models, aside from architecture, is the way that the model outputs are used to compute the loss.

The rate decoded models are trained to target an 80% firing rate for the correct output neuron and a 20% firing rate for all incorrect output neurons, which are standard target values in snnTorch implementations [19]. The firing rates represent the percentage of timesteps that an output neuron should fire. The normalised firing rate is used to calculate the MSE loss (equation (4)).

The latency decoded models are trained to fire a spike in the correct output neuron at the earliest possible timestep and for the incorrect output neurons at the last possible timestep. The normalised timing relative to the duration of the input sample is used to calculate the MSE loss (equation (5)).

The loss for the SNN-HDC is calculated using MSE in equation (12) given N as the number of samples, H as the output hypervector and C as the target class hypervector. H is the sum of the output spikes with the desired 'on' dimensions clamped to 1 as shown in algorithm 1. The proposed hypervector generation process is based on the presence and absence of spikes. The SNN-HDC model has D output neurons mapped to a hypervector with dimensionality of D. The ith output neuron corresponds to the ith dimension in the hypervector. Every dimension in the hypervector is initialised with 0. Should the ith output neuron spike during the input of a data sample, the corresponding ith dimension will be flipped from 0 to 1. If the dimensions in a hypervector are considered as temporal feature flags then a dimension spiking multiple times means that specific temporal feature was observed multiple times. For a dimension where the target is a 1, loss does not increase for a spike count above one. For a dimension where the target is a 0, loss increases proportionally to the spike count,

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(H_i - C_i)^2. \tag{12}$$

4.4. Synaptic operations (SOPs) and estimated energy consumption

A common goal of neuromorphic hardware is consuming energy relative to the number of SOPs that take place. A SOP corresponds to a source neuron sending a spike event to a target neuron via a unique (non-zero) synapse [54]. The TrueNorth neuromorphic hardware uses 26 pJ per SOP [54]. The energy

Algorithm 1. Calculating loss of SNN-HDC model.

$H \leftarrow \sum(\textit{spikes output by each neuron})$
$C \leftarrow$ Target class hypervector
for $\textit{index}$ **in** H **do**
  if $C[\textit{index}] = 1$ **then**
    if $H[\textit{index}] > 1$ **then**
      $H[\textit{index}] \leftarrow 1$
    end if
  end if
end for
$\textit{Loss} \leftarrow \textit{MSE}(H, C)$

values shown here are estimates of the energy consumption of each model per sample of data that it processes. Every test set sample for a given dataset is processed by the trained SNNs. The total number of SOPs occurring in each SNN is recorded and the average number of SOPs per sample is calculated. This SOP value is multiplied by 26 pJ to produce an energy consumption estimate.

4.5. Membrane potential decay rate and dimensionality

The rate at which the membrane potential decays in a neuron influences the overall activity in the network. As the optimal decay rate may be different for different decoding techniques, every decoding technique is trained using multiple values and the optimal value for a given architecture is used for the final analysis. The membrane potential decay rate is defined as β in equation (10). Values of β tested are 0.5, 0.7, and 0.9. The values of 0.5 and 0.9 were selected as they are commonly seen β values in snnTorch [19] implementations, with 0.7 chosen as an intermediate value.

The dimensionality value used by the models is chosen experimentally by analysing the accuracy and SOPs of various dimensionality values. The dimensionality values tested range from 11 (giving the SNN-HDC model the same number of output neurons as the rate and latency models for the DvsGesture dataset [36]) up to a value where the accuracy results no longer benefit from higher dimensionality.

The DvsGesture dataset [36] serves as an exploratory stage to determine an optimal combination of membrane potential decay rate and dimensionality. These values are then fixed for the SL-Animals-DVS dataset [48].

4.6. Measuring inference latency

The inference latency of the models quantifies how quickly classifications are made. As three different decoding methods are evaluated, three different ways of measuring inference latency must be used. For rate decoding the inference latency of an output is defined as how long it takes the network to reach a softmax of at least 0.99 across the sum of the output neuron spikes, as suggested by recent literature [55]. For latency decoding, the inference latency is defined as the time required for the network to output its first spike. For hyperdimensional decoding the inference latency is defined as the earliest timestep at which the predicted class does not change for the remaining duration of the input sample. Inference latency is calculated by passing all test samples into the trained networks, and reported as average $\pm$ standard deviation, for each dataset.

4.7. Identifying unknown classes

Unlike rate and latency decoded models, the SNN-HDC is not inherently constrained to a fixed number of output classes. A rate or latency decoded SNN with eleven output neurons can only represent eleven classes. As the SNN-HDC outputs hypervectors instead of classes, regardless of how many classes it has been trained on, new classes can be identified if the output hypervector exhibits low similarity with all known class hypervectors. This ability to detect unknown classes is explored using the DvsGesture dataset [36]. This dataset contains eleven different classes, ten of which are semantically assigned concepts and one of which is a random concept. After the optimal β and dimensionality parameters have been chosen for the eleven class SNN-HDC, the same parameters are used to train another SNN-HDC model on a subset of the dataset containing only the ten assigned concepts. This model is then evaluated on all eleven concepts, identifying the random concepts based on their dissimilarity to the known class hypervectors.

The eleven class SNN-HDC models classify inputs based just on similarity. The class hypervector with the highest similarity is chosen as the prediction. The ten class SNN-HDC model will classify inputs based on a similarity threshold. The ten class SNN-HDC model will classify an input as a known

class if the normalised Hamming distance is less than δ, otherwise, the input will be classified as an unknown class. The values of δ used for analysis range from 0.1 to 0.4, representing high alignment and low alignment. The accuracy for both the whole dataset as well as just the eleventh class is tracked. The performance of the ten class SNN-HDC is also compared to the eleven class SNN-HDC.

5. Results

This section presents the results of the experiments outlined in the methodology section. An analysis of the SNN-HDC models is performed in subsection 5.1, followed by an analysis of the rate and latency decoded models in subsection 5.2. Based on this analysis, one model is selected for each of the five model types. Model activity, including the number of spikes, estimated energy consumption and inference latency is then compared in subsections 5.3 and 5.4. The ability of the SNN-HDC model to identify classes that it has not been trained on is evaluated in subsection 5.5. Comparisons with existing SNN and HDC approaches are made in subsection 5.6. Detailed exploratory experimentation of the DvsGesture dataset [36] is presented first to determine optimal parameters, with subsequent datasets only reporting the key findings. Finally, the models produced in this research are compared to models from other research that have been trained on the same dataset in subsections 5.7 and 5.8.

5.1. SNN-HDC analysis

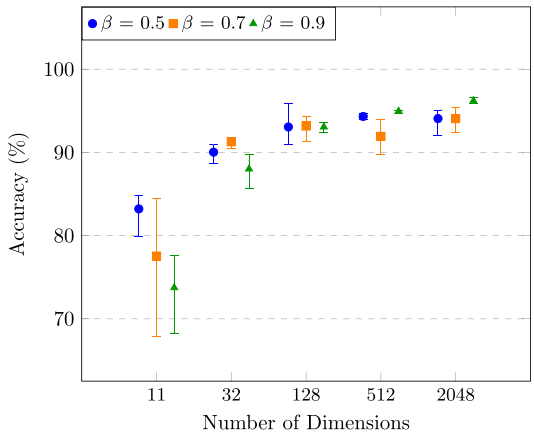
Figure 4(a) shows the accuracy of SNN-HDC models trained using different dimensionality and β values. In general increasing the number of dimensions leads to a higher classification accuracy. The highest accuracy for 11 dimensions is 84.85% ($\beta = 0.5$), whereas the highest accuracy for 2048 dimensions is 96.59% ($\beta = 0.9$). This is expected as higher dimension hypervectors are more robust and can contain a higher number of errors before losing their meaning. The value of β does not affect accuracy significantly but it does affect the consistency of the accuracy achieved beyond 32 dimensions, where $\beta = 0.9$ offers better consistency than other values. The trends indicate that all three β values tested reached a plateau with increasing dimensionality values.

Figure 4(b) shows the estimated energy used by various SNN-HDC models trained using different dimensionality and β values. Details of how energy consumption is estimated are provided in section 4.4. Increasing the number of dimensions results in higher estimated energy consumption. This behaviour is expected as higher dimensionality values require more neurons and synapses. A prominent trend is the way the estimated energy consumption increases for increased dimensionality values. For $\beta = 0.5$ and $\beta = 0.7$ the estimated energy consumption increases exponentially. When $\beta = 0.5$, 11 dimensions consume an average of 2.47 mJ and 2048 dimensions consume an average of 6.79 mJ. For $\beta = 0.9$ the estimated energy consumption increases linearly, although dimensionality values beyond 2048 may show that the trend is still exponential, albeit at a reduced rate. When $\beta = 0.9$, 11 dimensions consume an average of 2.42 mJ and 2048 dimensions consume an average of 2.96 mJ. The general trend of higher β values using less estimated energy than lower β values is expected. A lower β value causes the membrane voltage to decay quicker meaning the neuron can only integrate information from a shorter time period. As input spikes need to be temporally close to each other to cause a neuron to spike, during training the model compensates for this by making the input neurons spike more frequently, thereby increasing energy consumption.

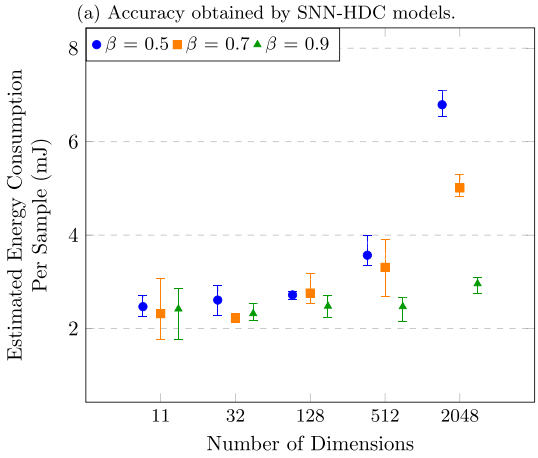
While the plots in figure 4 only show a subset of dimensions, more dimensions were evaluated during experimentation. The dimensions plotted are selected to illustrate the trend. Experimental results indicate that the accuracy of the SNN-HDC model did not improve beyond 1024 dimensions. Although the accuracy did not increase with higher dimensions, the estimated energy consumption did. The SNN-HDC model used for further analysis in this paper is therefore configured with a dimensionality of 1024 and a β value of 0.9. For fair comparison, the additional hidden layer in the deeper versions of the rate and latency decoded SNNs will contain 1024 neurons, equal to the dimensionality value of the chosen SNN-HDC model.

5.2. Rate and latency SNN analysis

Figure 5(a) shows the accuracy achieved by the Rate-1, Rate-2, Latency-1 and Latency-2 models on the DvsGesture dataset [36]. The rate decoded models achieve higher accuracy than the latency decoded models. The most accurate rate decoded SNN (Rate-2, $\beta = 0.9$) achieved 97.35%, whereas the most accurate latency decoded SNN (Latency-2, $\beta = 0.9$) achieved 85.23%. This is expected as rate decoding typically outperforms latency decoding in terms of classification accuracy. The deeper versions of both decoding methods consistently achieve higher accuracy than their shallower counterparts. This is expected as deeper networks with more parameters typically attain higher accuracy. The value of β has



(a) Accuracy obtained by SNN-HDC models.



(b) Estimated energy consumption in SNN-HDC models.

Figure 4. SNN-HDC results trained on the DvsGesture dataset [36] for various dimensionality and membrane potential decay rate (β) values.

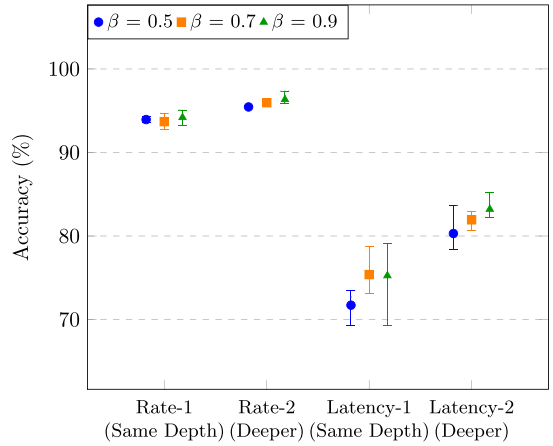
negligible effect on the accuracy of the rate decoded models, however it has significant impact on the accuracy of the latency decoded models, with higher values attaining higher accuracy. A possible explanation for this is that rate decoding, unlike latency decoding, is not dependent on the timing of individual spikes but rather the accumulation of many spikes. The latency decoded models have higher ranges of accuracy compared to the rate decoded models likely due to latency decoding being unstable due to its reliance on a single spike for classification.

Figure 5(b) shows the estimated energy consumption of the Rate-1, Rate-2, Latency-1 and Latency-2 models on the DvsGesture dataset [36]. Details of how energy consumption is estimated are seen in section 4.4. The rate decoded models used more estimated energy than the latency decoded models which is expected as the higher energy consumption of rate decoding compared to latency decoding is typically considered one of its drawbacks. When $\beta = 0.9$, Rate-2 consumes an average of 9.67 mJ, whereas Latency-2 consumes 6.88 mJ. The deeper version of each model used more estimated energy than its shallower counterpart which is expected as the deeper models contain more neurons and synapses. The value of β has negligible impact on estimated energy consumption in all models.

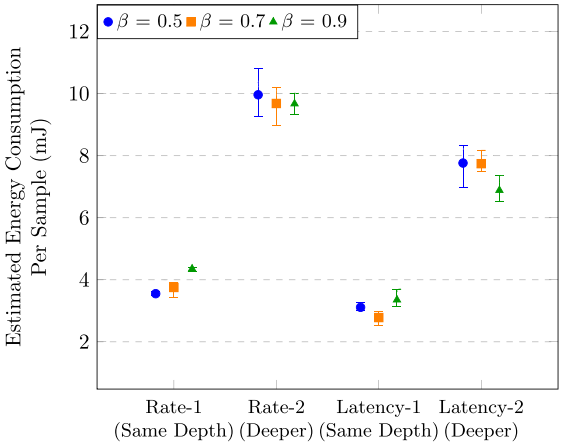
5.3. Model activity

The highest accuracy model from each of the five categories is selected for further analysis. For the SNN-HDC model this is a dimensionality of 1024 and a β value of 0.9. For the Rate-1 SNN this is a β value of 0.9. For the Rate-2 SNN this is a hidden layer size of 1024 neurons and a β value of 0.9. For the Latency-1 SNN this is a β value of 0.9. For the Latency-2 SNN this is a hidden layer size of 1024 and a β value of 0.9. A comparison of relevant metrics for all models is shown in table 6.

Figure 6(a) shows the per sample average number of spikes fired by each layer for all five models when passed the entire DvsGesture [36] test set. The first two layers of each model are structurally identical, however, the SNN-HDC model fired significantly fewer spikes than all other models. In Layer

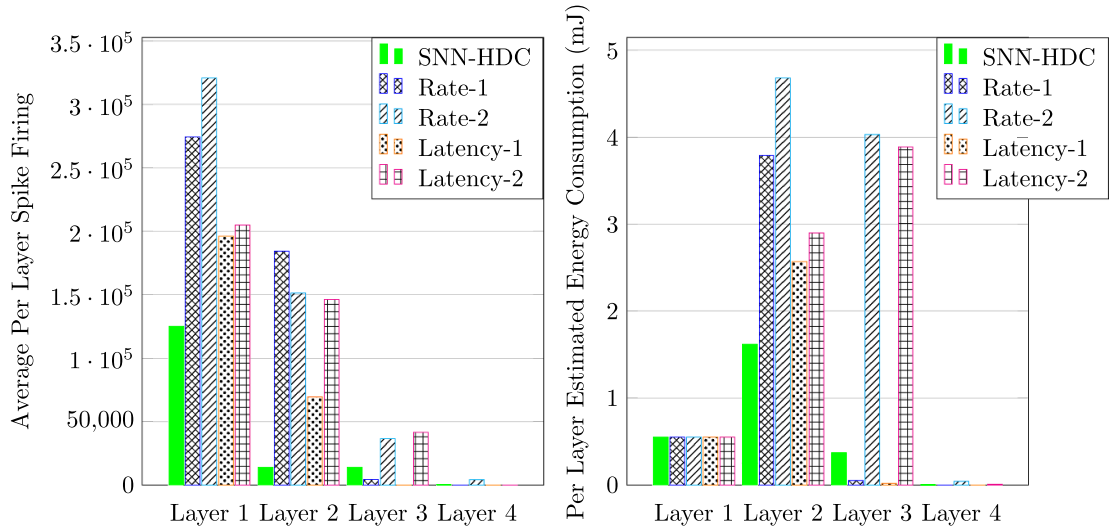


(a) Accuracy obtained by comparable rate and latency decoded models.



(b) Estimated energy consumption in comparable rate and latency decoded models.

Figure 5. Rate and latency decoded results trained on the DvsGesture dataset [36] for various membrane potential decay rates (β).



(a) Average per layer spikes fired in the highest accuracy models.

(b) Per layer estimated energy consumption in the highest accuracy models.

Figure 6. SNN-HDC, rate decoded and latency decoded layer activity. Numbers obtained over DvsGesture [36] test set.

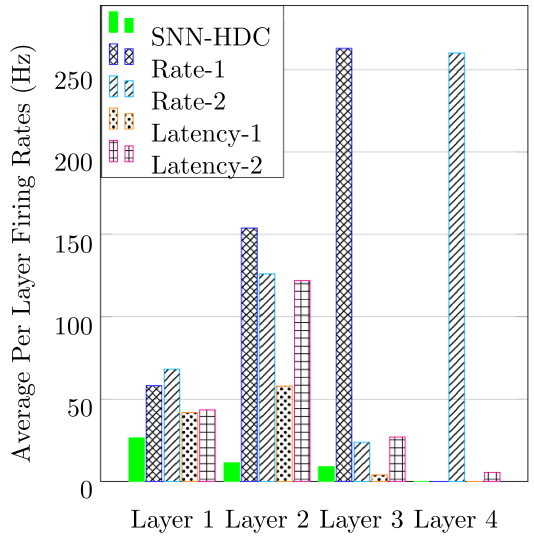


Figure 7. Average per layer firing rate of neurons in each model per sample of the DvsGesture [36] test set.

1, the SNN-HDC model fired 1.57 × to 2.56 × fewer spikes compared to the other models. In Layer 2, the SNN-HDC model fired 5.05 × to 13.4 × fewer spikes compared to the other models. In Layer 3 the structure of the models begins to differ, with Rate-1 and Latency-1 containing 11 neurons and the SNN-HDC, Rate-2 and Latency-2 containing 1024 neurons. Comparing Layer 3 of the SNN-HDC to Rate-1 and Latency-1 shows firing count increases of 3.18 × and 209 × respectively. This is expected due to the SNN-HDC model containing significantly more neurons in that layer. Comparing the SNN-HDC to Rate-2 and Latency-2 shows firing count decreases of 2.64 × and 3.02 × respectively. Although the SNN-HDC fired more spikes than Rate-1 and Latency-1 in Layer 3, this layer does not make a significant impact to the overall spike count of the models. Comparing the total spike counts of each model shows that the SNN-HDC fired 1.74 × to 3.36 × fewer spikes overall. The rate decoded models fire more spikes than the SNN-HDC as the rate decoded loss function necessitates significantly more output spikes than the hyperdimensional loss function. The latency decoded models fire more spikes than the SNN-HDC, possibly because a large number of spikes over a relatively short time period within the network are needed to help differentiate between classes when using a loss function based around the earliest possible firing of a single output spike.

Figure 6(b) shows the estimated energy consumption in each layer of all five models. The energy consumption is determined by the number of spikes processed and the architecture of the network. In Layer 1, the models all consume the same amount of energy as the first layer in all models has the same structure and takes the same inputs. In Layer 2, the SNN-HDC model consumed between 1.59 × and 2.89 × less energy. This is expected as this layer has a common structure between all models and the SNN-HDC had to accumulate the least number of spikes. In Layer 3, compared to Rate-1 and Latency-1, the SNN-HDC consumed 7.40 × and 18.5 × more energy respectively. This is expected as the size of the layers is significantly different. In Layer 3, compared to Rate-2 and Latency-2, the SNN-HDC consumed 10.9 × and 10.5 × less energy respectively. Comparing the total estimated energy consumption of each model shows that the SNN-HDC consumed between 1.24 × and 3.67 × less energy overall.

Figure 7 shows the average per sample firing rate (Hz) of all models per layer. The SNN-HDC exhibits a lower firing frequency than all other models in every layer, except one model-layer combination (Layer 3 of Latency-1). In Layer 1, the SNN-HDC reduced firing frequency by 1.57 × and 2.56 ×. In Layer 2, the SNN-HDC reduced firing frequency by 5.04 × and 13.4 ×. In Layer 3, barring the Latency-1 model, the SNN-HDC reduced firing frequency by 2.63 × and 29.2 ×. Comparing Layer 3 of the SNN-HDC and Latency-1 models shows that the SNN-HDC exhibited a firing frequency increase of 2.25 ×. This relative increase for this specific model and layer is the result of the latency decoded loss function which causes few output spikes to be fired. Table 3 reports the average per sample firing rate (Hz) of each model as a whole. The rate models have the highest firing rate, followed by the latency models and then the SNN-HDC. The SNN-HDC has the lowest overall firing rate, with reductions ranging from 2.19 × to 3.81 ×.

Table 3. Average firing rate of neurons in each model per sample of the DvsGesture [36] test set.

Model	Number of neurons	Average firing rate (Hz)	Relative firing rate
SNN-HDC	4960	20.5	1 ×
Rate-1	3947	78.2	3.81 ×
Rate-2	4971	68.2	3.33 ×
Latency-1	3947	44.9	2.19 ×
Latency-2	4971	52.7	2.57 ×

Table 4. Inference latency of models on DvsGesture [36] test set shown as average ± standard deviation.

Model	Inference latency	Relative
SNN-HDC	162 ms ± 278 ms	1 ×
Rate-1	208 ms ± 299 ms	1.28 ×
Rate-2	133 ms ± 218 ms	0.82 ×
Latency-1	206 ms ± 311 ms	1.27 ×
Latency-2	188 ms ± 234 ms	1.16 ×

5.4. Model inference latency

Table 4 shows the inference latency achieved by each of the five models on the DvsGesture [36] test set. The inference latency is evaluated according to section 4.6, and reported as average ± standard deviation in milliseconds. Although latency decoding is typically expected to exhibit improved inference latency compared to rate decoding, the results here show this is not always the case. Whilst this appears inconsistent given how the loss functions are defined (equations (4) and (5)), it is not unexpected as prior research has already shown that rate decoding can attain better inference latency than latency decoding when trained on neuromorphic datasets [22].

Both deeper models, exhibit lower inference latency than their shallower counterparts. This is possibly due to the additional hidden layer increasing class separability, as evidenced by both deeper models also having better accuracy than their shallower counterparts (table 6). The inference latency of the SNN-HDC is better and more consistent than Rate-1, but it is worse and less consistent than Rate-2. This again is likely due to structural differences, with the deeper networks achieving higher accuracy due to increased class separability.

If a rate decoded model has been trained to successfully separate classes, it follows from the loss function definition (see equation (4)) that it will fire more output spikes in the correct neuron than in the incorrect neurons. This allows the deeper network to stabilise onto the correct answer quicker. The inference latency of rate decoding is defined as the time it takes for the softmax of the sum of the output spikes to reach a threshold of 0.99, as discussed in section 4.6. With this method of calculating inference latency, a rate decoded model will have a lower inference latency if one of its output neurons is firing significantly more than all other output neurons, which is the expected activity for Rate-2. It is therefore observed generally that SNN structure and decoding method combine to impact the internal spiking activity and inference latency.

5.5. Unknown class detection

The eleventh class in the DvsGesture dataset [36] is the random gesture. This class was removed from both the training and testing datasets. The SNN-HDC model trained here has the same parameters as the SNN-HDC model selected from the full dataset. The dimensionality is 1024 and the β value is 0.9. The model was trained across three different runs which affected weight initialisation and generated class hypervectors. The average classification accuracy was 96.67% with a range of $\pm 0.83\%$ when classifying only the ten classes that were used for training. The highest accuracy of these models was 97.08%, which is used for further analysis in this section.

Figure 8 shows the accuracy obtained by the SNN-HDC model trained on only 10 classes, but used to classify all 11 classes. The δ value refers to normalised Hamming distance (equation (3)). Classifications of known classes are only considered correct if they are both accurate and have a normalised Hamming distance that is less than δ. Classifications of the unknown class are only considered correct when the output has a normalised Hamming distance of more than δ to every known class. The full dataset accuracy shows the accuracy obtained by the model given both of these constraints. The unknown class accuracy only shows the accuracy of identifying the unknown class. As δ decreases, the model's ability to detect the unknown class improves. When $\delta = 0.4$ the model is unable to detect any

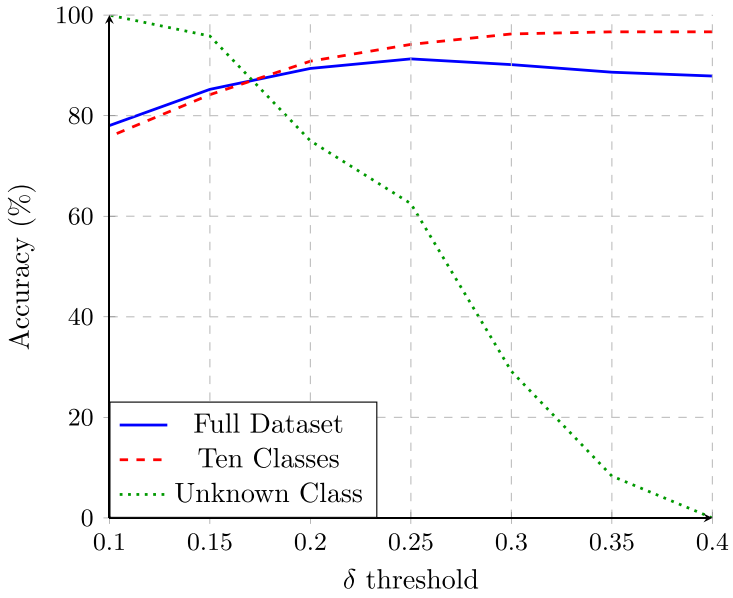


Figure 8. Accuracy obtained by the 10-class trained SNN-HDC when performing classification on the full 11 class DvsGesture dataset [36]. A known classification is correct if its normalised Hamming distance is less than δ to the correct class hypervector. An unknown classification is correct if its normalised Hamming distance is more than δ to all known class hypervectors.

unknown samples. When $\delta = 0.1$ the model can detect every unknown sample. Lowering the value of δ below 0.25 has a negative impact on the full dataset accuracy. This is because the requirements of the output are more strict and necessitate fewer errors. $\delta = 0.25$ provides a balanced trade-off as it can detect 62.5% of unknown samples while still achieving a ten class accuracy of 94.17%.

These results indicate that while it is better to train a model on all 11 classes, training on only 10 classes still results in high performance. With an appropriate δ value, the SNN-HDC model is capable of classifying known and unknown samples to such a high accuracy that it outperforms SQUAT [35] (the study which inspired the architecture of the models developed here) as well as both state-of-the-art SNN and HDC combinations [37, 40] reviewed in section 3.

5.6. Comparisons with existing SNN and HDC approaches

Table 5 shows how the proposed SNN-HDC compares to the related SNN and HDC approaches, namely SpikeHD [37] and HyperSpike [40] (see section 3). The results presented are for the DvsGesture dataset [36]. The SNN-HDC achieves 8.79% higher accuracy than SpikeHD [37] and 9.39% higher accuracy than HyperSpike [40]. This accuracy increase is likely due to the SNN-HDC being directly trained to output binary hypervectors, unlike SpikeHD [37], which first trained the SNN with a different decoding method, or HyperSpike [40], which did not train the SNN at all. This different training methodology is likely what allows the SNN-HDC to more efficiently generate hypervectors than SpikeHD [37] and HyperSpike [40]. This approach also enables more effective dimension use which lowers the overall number of dimensions required. The SNN-HDC uses a dimensionality of 1024, SpikeHD [37] used a dimensionality of 4000, and HyperSpike [40] used a dimensionality of 10 000. The SNN-HDC therefore requires less memory for the same number of class hypervectors compared to both SpikeHD [37] and HyperSpike [40].

The SNN-HDC uses the presence and absence of output spikes for hypervector generation as opposed to the projection matrices used by SpikeHD [37] and HyperSpike [40]. This difference in hypervector generation allows the processing of the SNN-HDC to occur per-timestep as opposed to per-sample like SpikeHD [37] and HyperSpike [40]. The per-timestep processing of the SNN-HDC subsequently allows it to achieve lower inference latency than both SpikeHD [37] and HyperSpike [40]: SNN-HDC can perform inference during a sample input; while both SpikeHD [37] and HyperSpike [40] can only perform inference after a complete sample has been input.

The capability for unknown class detection was also tested here and the SNN-HDC was able to detect 100% of unknown classes from the DvsGesture dataset [36]. While this ability was not tested in either SpikeHD [37] or HyperSpike [40], it is possible that both of these works are capable of this given that both works produced models that output hypervectors for comparison.

Table 5. Comparison between the proposed SNN-HDC, and the related state-of-the-art SNN and HDC combinations: SpikeHD [37] and HyperSpike [40]. Results shown are for the DvsGesture dataset [36].

Model	Parameter number	Accuracy	Hypervector generation method	Dimensions	Inference latency	Unknown class detection	Processing type
SNN-HDC	834 k	96.59%	Presence/absence	1024	During sample input	100%	Per-timestep
SpikeHD [37]	968 k	87.8%	Projection matrix	4000	After sample input	Untested	Per-sample
HyperSpike [40]	—	87.2%	Projection matrix	10000	After sample input	Untested	Per-sample

Table 6. Comparison between the proposed SNN-HDC, two rate decoded SNNs and two latency decoded SNNs. Accuracy, spike count and estimated energy consumption were obtained after each of the networks trained to convergence on the DvsGesture dataset [36].

Model	Parameter number	Accuracy	Average spike count	Relative spike count	Estimated energy	Relative estimated energy	Inference latency	Relative inference latency
SNN-HDC	834 k	96.59%	1.53×10^5	1 ×	2.54 mJ	1 ×	162 ms	1 ×
Rate-1 (Same depth)	22.5 k	95.08%	4.63×10^5	3.03 ×	4.39 mJ	1.73 ×	208 ms	1.28 ×
Rate-2 (Deeper)	845 k	97.35%	5.13×10^5	3.36 ×	9.31 mJ	3.67 ×	133 ms	0.82 ×
Latency-1 (Same depth)	22.5 k	79.17%	2.66×10^5	1.74 ×	3.14 mJ	1.24 ×	206 ms	1.27 ×
Latency-2 (Deeper)	845 k	85.23%	3.93×10^5	2.57 ×	7.37 mJ	2.90 ×	188 ms	1.16 ×

5.7. Overall DvsGesture results

Table 6 shows the performance of all models trained here in terms of accuracy achieved, estimated energy consumption and inference latency. The SNN-HDC achieves at least a 1.51% higher accuracy than Rate-1, Latency-1 and Latency-2 but achieves 0.76% less than Rate-2. The SNN-HDC exhibits the smallest spike count of all the models, with reductions ranging from 1.74 × to 3.36 ×. The SNN-HDC has the smallest energy consumption, with reductions ranging from 1.24 × to 3.67 ×. The SNN-HDC achieves at least a 1.16 × lower inference latency than Rate-1, Latency-1 and Latency-2 but has 1.22 × higher inference latency than Rate-2.

Table 7 compares the models tested here with other works in the literature. Rate-1 and the SQUAT model [35] have the same architecture, however Rate-1 is trained using higher temporal resolution. Rate-1 achieves an 8.84% higher accuracy than SQUAT [35]. This indicates a low temporal resolution diminishes temporal information hindering performance, as discussed in section 2.6. Table 7 also shows that the SNN-HDC model achieves a higher accuracy than the related SNN and HDC models, achieving 8.79% more than SpikeHD [37] and 9.39% more than HyperSpike [40].

5.8. SL-Animals results

Table 8 reports resulting metrics for all five models trained on the SL-Animals-DVS dataset [48]. Table 9 compares the performance of the models in this work to other published works. All networks were trained with $D = 1024$ and $\beta = 0.9$, as determined by the experiments performed on the DvsGesture dataset [36]. While the trend of rate decoded models achieving higher accuracy than latency decoded models remains, not all other previously observed trends remain. Here the SNN-HDC is the highest accuracy model, possibly attributed to the robustness of hypervectors. The trend of deeper models achieving higher accuracy than their shallower counterparts does not hold true for the latency decoded networks. This is likely the result of structural differences which appear in the fully-connected portion of the networks. Latency-1 employs a fully-connected structure of [25 - 19]. Latency-2 employs a fully-connected structure of [25 - 1024 - 19]. The full architectures are shown in table 2. The increased number of neurons feeding into the output layer may cause instability for latency decoding. This was not observed for the DvsGesture dataset [36] where there was a much smaller structural difference between Latency-1 ([800 - 11]) and Latency-2 ([800 - 1024 - 11]).

The SNN-HDC exhibits the lowest average spike count, with reductions ranging from 1.36 × to 2.70 ×. The SNN-HDC also exhibits the lowest energy consumption, with reductions ranging from 1.38 × to 2.27 ×. These results show the energy savings of the SNN-HDC generalise across the architectures and datasets evaluated.

The inference latencies of all models here are higher than the inference latencies for the DvsGesture dataset [36] seen in table 4. The difference could be attributed to the context of the actions being performed, with DvsGesture [36] samples taking less time to become distinct than SL-Animals-DVS [48] samples. The latency decoded models have the lowest averages, however, this is negated by their lower

Table 7. Accuracy obtained by the models in this work (top) and other SNN models (bottom) trained on the DvsGesture dataset [36].

Model	Parameter number	Model type	Decoding method	Accuracy
Rate-2	845 k	SNN	Rate	97.35%
SNN-HDC	834 k	SNN	HDC	96.59%
Rate-1	22.5 k	SNN	Rate	95.08%
Latency-2	845 k	SNN	Latency	85.23%
Latency-1	22.5 k	SNN	Latency	79.17%
ACE-BET [56]	11.7 M	ANN	—	98.88%
mMND [21]	1.1 M	SNN	Rate	98.0%
DECOLLE [38]	1.64 M	SNN	Rate	95.54%
SLAYER [47]	1.06 M	SNN	Rate	93.64%
FS [22]	740 k	SNN	Latency	92.8%
SpikeHD [37]	968 k	SNN	HDC	87.8%
HyperSpike [40]	—	SNN	HDC	87.2%
SQUAT [35]	22.5 k	SNN	Rate	86.24%

Table 8. Comparison between the proposed SNN-HDC, two rate decoded SNNs and two latency decoded SNNs. Accuracy, spike count and estimated energy consumption were obtained after each of the networks trained to convergence on the SL-Animals-DVS dataset [48].

Model	Parameter number	Accuracy	Average spike count	Relative spike count	Estimated energy	Relative estimated energy	Inference latency	Relative inference latency
SNN-HDC	55.9 k	74.13%	6.86×10^5	1 ×	3.61 mJ	1 ×	570 ms	1 ×
Rate-1 (Same depth)	29.8 k	70.68%	1.85×10^6	2.70 ×	8.21 mJ	2.27 ×	668 ms	1.17 ×
Rate-2 (Deeper)	75.4 k	72.74%	1.71×10^6	2.49 ×	7.56 mJ	2.09 ×	592 ms	1.04 ×
Latency-1 (Same depth)	29.8 k	47.34%	9.31×10^5	1.36 ×	4.97 mJ	1.38 ×	361 ms	0.63 ×
Latency-2 (Deeper)	75.4 k	45.12%	1.08×10^6	1.57 ×	5.43 mJ	1.50 ×	356 ms	0.62 ×

Table 9. Accuracy obtained by the models in this work (top) and other SNN models (bottom) trained on all subsets of the SL-Animals-DVS dataset [48].

Model	Parameter number	Model type	Decoding method	Accuracy
SNN-HDC	55.9 k	SNN	HDC	$74.13\% \pm 4.26\%$
Rate-2	75.4 k	SNN	Rate	$72.74\% \pm 3.73\%$
Rate-1	29.8 k	SNN	Rate	$70.68\% \pm 4.00\%$
Latency-1	29.8 k	SNN	Latency	$47.34\% \pm 3.66\%$
Latency-2	75.4 k	SNN	Latency	$45.12\% \pm 4.46\%$
EventRPG [49]	11.7 M	SNN	Rate	91.59%
EvT [50]	0.48 M	ANN	—	88.12%
SL-Animals [48]	1.37 M	SNN	Rate	$70.6\% \pm 7.8\%$

classification accuracy. The SNN-HDC achieves lower inference latency than both rate decoded models, with reductions of 1.17 × and 1.04 × over Rate-1 and Rate-2 respectively.

6. Discussion and conclusion

This work presents an SNN decoding method inspired by distributed representations through the combination of SNNs and HDC. The proposed method is evaluated and compared to rate and latency decoded SNNs trained on neuromorphic datasets. For comparable rate and latency decoded models, the SNN-HDC maintains or improves upon the accuracy achieved, attains generally lower inference latency, requires fewer spikes and consumes less estimated energy. On the DvsGesture dataset [36], the SNN-HDC achieved reductions of 1.74 × to 3.36 × for spike count and reductions of 1.24 × to 3.67 × for estimated energy consumption. On the SL-Animals-DVS dataset [48], the SNN-HDC achieved reductions of 1.36 × to 2.70 × for spike count and reductions of 1.38 × to 2.27 × for estimated energy consumption. The SNN-HDC also offers the ability to detect unknown classes that it has not been trained on, being able to detect 100% of the samples from an unseen/untrained class from the DvsGesture dataset [36].

The proposed SNN-HDC model extends prior SNN and HDC approaches, SpikeHD [37] and HyperSpike [40], in several aspects. The SNN-HDC is trained directly to produce hypervectors, eliminating the need to initially train the network using other decoding methods. The hypervector generation process is modified, removing the reliance on matrix multiplication, saving energy by reducing the amount of computation used. The SNN-HDC also lowers inference latency, as hypervectors are built up over time and can be classified at any point in time rather than only at the end of a sample. Compared to SpikeHD [37] and HyperSpike [40], the SNN-HDC uses a more energy efficient method for hypervector comparison based on Hamming distance.

Although the SNN-HDC approach offers several advantages over rate and latency decoding, these benefits come with potentially increased memory consumption when directly replacing the output layer. The SNN-HDC required 37.1 × and 1.88 × more parameters than the same depth one-hot decoded counterpart on the DvsGesture [36] and SL-Animals-DVS [48] datasets respectively. The higher memory footprint can be mitigated by acting on the number of neurons in the layer before the output layer. When the output layer of a network is removed and its penultimate layer is decoded with hyperdimensional decoding, as is the case between the SNN-HDC model and its deeper one-hot decoded counterparts, the SNN-HDC model requires fewer parameters. The SNN-HDC model required 1.01 × and 1.35 × fewer parameters than the deeper one-hot decoded counterpart on the DvsGesture [36] and SL-Animals-DVS [48] datasets respectively. When the number of classes exceeds approximately 2633, the SNN-HDC model is expected to use fewer parameters than a one-hot decoded counterpart as the expressivity of binary hypervectors will exceed the expressivity of one-hot decoding, as discussed in section 2.4.

This work further demonstrates the potential of combining SNNs with HDC and warrants continued exploration. Future research on this topic could investigate hypervector generation processes that include the number of spikes and inter-spike interval in the dimensional values to increase the information obtained per output. This could improve classification accuracy and reduce the energy consumption of the network. The effect of the number of active bits in the class hypervectors could also be analysed, as using fewer active bits may further reduce energy consumption. The scalability of the SNN-HDC approach could be explored when applied to problems or datasets with such a high number of classes that hypervectors are more expressive than one-hot decoding, as discussed in section 2.4. The use of high dimension hypervectors may result in significant memory requirement reductions. The SNN could also be trained to output dynamic rather than static hypervectors, producing an output that moves over time through hyperspace. Dynamic hypervectors could enable continuous classifications over time in an event driven manner, without requiring network resets. This opens up applications to continuous online data streams, which would be well suited for neuromorphic deployment.

In summary, this paper presents an SNN specific decoding methodology and model called SNN-HDC. The SNN-HDC demonstrates several desirable advantages over existing approaches, making it a compelling alternative to rate and latency decoding. With the results presented in this paper and the significant potential for future work, the SNN-HDC is positioned well to support the development of novel neuromorphic systems and establish new research. It is hoped that this approach and the potential new research directions it brings will help further establish SNNs as a low-power alternative in the quest of energy efficient artificial intelligence.

Data availability statement

The data that support the findings of this study are openly available at the following URL/DOI: https://doi.org/10.5281/zenodo.17522304 [57].

Funding

This research was supported through the NimbleAI project, funded via the Horizon Europe Research and Innovation programme (Grant Agreement 10 107 0679), and UKRI under the UK government's Horizon Europe funding guarantee (Grant Agreement 10 039 070); the Horizon Europe AIDA4Edge project (Grant Agreement 10 116 0293); and the EPSRC Edgy Organism project (EP/Y030133/1).

Author contributions

Cedrick Kinavuidi 0009-0001-2860-9163
Conceptualization (equal), Data curation (lead), Formal analysis (lead), Investigation (lead), Methodology (lead), Project administration (equal), Resources (equal), Software (lead), Validation (lead), Visualization (lead), Writing – original draft (lead), Writing – review & editing (equal)

Luca Peres 0000-0001-9748-9073
Conceptualization (equal), Formal analysis (supporting), Funding acquisition (supporting), Investigation (supporting), Methodology (supporting), Project administration (equal), Resources (equal), Software (supporting), Supervision (supporting), Validation (supporting), Visualization (supporting), Writing – original draft (supporting), Writing – review & editing (equal)

Oliver Rhodes 0000-0003-1728-2828
Conceptualization (equal), Formal analysis (supporting), Funding acquisition (lead), Investigation (supporting), Methodology (supporting), Project administration (equal), Resources (equal), Software (supporting), Supervision (lead), Validation (supporting), Visualization (supporting), Writing – original draft (supporting), Writing – review & editing (equal)

References

[1] Maass W 1997 *Neural Netw.* **10** 1659–71
[2] Deng L, Wu Y, Hu X, Liang L, Ding Y, Li G, Zhao G, Li P and Xie Y 2020 *Neural Netw.* **121** 294–307
[3] Roy K, Jaiswal A and Panda P 2019 *Nature* **575** 607–17
[4] Yamazaki K, Vo-Ho V K, Bulsara D and Le N 2022 *Brain Sci.* **12** 863
[5] Tavanaei A, Ghodrati M, Kheradpisheh S R, Masquelier T and Maida A 2019 *Neural Netw.* **111** 47–63
[6] Attwell D and Laughlin S B 2001 *J. Cerebral Blood Flow Metab.* **21** 1133–45
[7] Caccavella C, Paredes-Vallés F, Cannici M and Khacef L 2024 Low-power event-based face detection with asynchronous neuromorphic hardware *2024 Int. Joint Conf. on Neural Networks (IJCNN)* pp 1–10 (available at: https://ieeexplore.ieee.org/abstract/document/10650843?signout = success)
[8] Schuman C D, Potok T E, Patton R M, Birdwell J D, Dean M E, Rose G S and Plank J S 2017 A survey of neuromorphic computing and neural networks in hardware (arXiv:1705.06963)
[9] Davies M, Wild A, Orchard G, Sandamirskaya Y, Guerra G A F, Joshi P, Plank P and Risbud S R 2021 *Proc. IEEE* **109** 911–34
[10] Guo W, Fouda M E, Eltawil A M and Salama K N 2021 *Front. Neurosci.* **15** 638474
[11] Schuman C, Rizzo C, McDonald-Carmack J, Skuda N and Plank J 2022 Evaluating encoding and decoding approaches for spiking neuromorphic systems *Proc. Int. Conf. on Neuromorphic Systems 2022* (ACM) pp 1–9
[12] Davidson S and Furber S B 2021 *Front. Neurosci.* **15** 651141
[13] Caporale N and Dan Y 2008 *Annu. Rev. Neurosci.* **31** 25–46
[14] Barrett D G T, Denéve S and Machens C K 2015 Optimal compensation for neuron death pages: 029512 section: New Results (available at: www.biorxiv.org/content/10.1101/029512v1)
[15] Jain A, Bansal R, Kumar A and Singh K D 2015 *Int. J. Appl. Basic Med. Res.* **5** 124
[16] Furber S 2012 *IEEE Spectr.* **49** 44–49
[17] Kanerva P 2009 *Cogn. Comput.* **1** 139–59
[18] Gerstner W, Kistler W M, Naud R and Paninski L 2014 *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition* 1st edn (Cambridge University Press) (available at: www.cambridge.org/core/product/identifier/9781107447615/type/book)
[19] Eshraghian J K, Ward M, Neftci E, Wang X, Lenz G, Dwivedi G, Bennamoun M, Jeong D S and Lu W D 2023 Training spiking neural networks using lessons from deep learning (arXiv:2109.12894 [cs])
[20] Auge D, Hille J, Mueller E and Knoll A 2021 *Neural Process. Lett.* **53** 4693–710
[21] She X, Dash S and Mukhopadhyay S 2021 Sequence approximation using feedforward spiking neural network for spatiotemporal learning: theory and optimization methods (available at: https://openreview.net/forum?id = bp-LJ4y_XC)
[22] Liu S, Leung V C H and Dragotti P L 2023 *Front. Neurosci.* **17** 1266003
[23] Yin B, Guo Q, Corradi F and Bohte S 2022 Attentive decision-making and dynamic resetting of continual running SRNNs for end-to-end streaming keyword spotting *Proc. Int. Conf. on Neuromorphic Systems 2022* ICONS '22 (Association for Computing Machinery) pp 1–8
[24] Wu D, Jin G, Yu H, Yi X and Huang X 2025 *Front. Neurosci.* **19** 1522788
[25] Klimo M, Lukáč P and Tarábek P 2021 *Appl. Sci.* **11** 3563
[26] Kleyko D, Rachkovskij D A, Osipov E and Rahimi A 2022 *ACM Comput. Surv.* **55** 1–40
[27] DeepSeek-AI, Liu A *et al* 2024 DeepSeek-V3 *Technical Report* version: 1 (arXiv:2412.19437 [cs])
[28] Leñero-Bardallo J A, Serrano-Gotarredona T and Linares-Barranco B 2011 *IEEE J. Solid-State Circuits* **46** 1443–55
[29] Liao F, Zhou F and Chai Y 2021 *J. Semiconduct.* **42** 013105
[30] Brandli C, Berner R, Yang M, Liu S C and Delbruck T 2014 *IEEE J. Solid-State Circuits* **49** 2333–41
[31] Chalich Y, Mallick A, Gupta B and Deen M J 2020 *PLoS One* **15** e0232788
[32] Abad G, Ersoy O, Picek S and Urbieta A 2024 Sneaky spikes: uncovering stealthy backdoor attacks in spiking neural networks with neuromorphic data *Proc. 2024 Network and Distributed System Security Symp.* (arXiv:2302.06279 [cs])
[33] Apolinario M P E and Roy K 2024 S-TLLR: STDP-inspired temporal local learning rule for spiking neural networks version: 4 (arXiv:2306.15220)
[34] Sun H, Cai W, Yang B, Cui Y, Xia Y, Yao D and Guo D 2023 A synapse-threshold synergistic learning approach for spiking neural networks version: 3 (arXiv:2206.06129 [cs])

[35] Venkatesh S, Marinescu R and Eshraghian J K 2024 SQUAT: stateful quantization-aware training in recurrent spiking neural networks *2024 Neuro Inspired Computational Elements Conf. (NICE)* pp 1–10 (available at: https://ieeexplore.ieee.org/abstract/document/10549198/figures#figures)
[36] Amir A *et al* 2017 A low power, fully event-based gesture recognition system *2017 IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)* pp 7388–97 (available at: https://ieeexplore.ieee.org/document/8100264)
[37] Zou Z, Alimohamadi H, Zakeri A, Imani F, Kim Y, Najafi M H and Imani M 2022 *Sci. Rep.* **12** 7641
[38] Kaiser J, Mostafa H and Neftci E 2020 *Front. Neurosci.* **14** 424
[39] Deng L 2012 *IEEE Signal Process. Mag.* **29** 141–2
[40] Morris J, Lui H W, Stewart K, Khaleghi B, Thomas A, Marback T, Aksanli B, Neftci E and Rosing T 2022 HyperSpike: hyperdimensional computing for more efficient and robust spiking neural networks *2022 Design, Automation & Test in Europe Conf. & Exhibition (DATE)* pp 664–9 (available at: https://ieeexplore.ieee.org/document/9774644)
[41] Orchard G, Jayawant A, Cohen G K and Thakor N 2015 *Front. Neurosci.* **9** 437
[42] Bi Y, Chadha A, Abbas A, Bourtsoulatze E and Andreopoulos Y 2019 Graph-based object classification for neuromorphic vision sensing *2019 IEEE/CVF Int. Conf. on Computer Vision (ICCV)* pp 491–501 (available at: https://ieeexplore.ieee.org/document/9010397)
[43] Galvez R L, Dadios E P, Bandala A A and Vicerra R R P 2019 YOLO-based threat object detection in x-ray images *2019 IEEE 11th Int. Conf. on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment and Management (HNICEM)* pp 1–5 (available at: https://ieeexplore.ieee.org/document/9073599)
[44] Li X, Wang W, Xue F and Song Y 2018 *Physica A* **491** 716–28
[45] Horowitz M 2014 1.1 Computing's energy problem (and what we can do about it) *2014 IEEE Int. Solid-State Circuits Conf. Digest of Technical Papers (ISSCC)* pp 10–14 (available at: https://ieeexplore.ieee.org/document/6757323)
[46] Lenz G, Chaney K, Shrestha S B, Oubari O, Picaud S and Zarrella G 2021 Tonic: event-based datasets and transformations (available at: https://zenodo.org/records/5079802)
[47] Shrestha S B and Orchard G 2018 SLAYER: spike layer error reassignment in time (arXiv:1810.08646 [cs])
[48] Vasudevan A, Negri P, Di Ielsi C, Linares-Barranco B and Serrano-Gotarredona T 2022 *Pattern Anal. Appl.* **25** 505–20
[49] Sun M, Zhang D, Ge Z, Wang J, Li J, Fang Z and Xu R 2024 EventRPG: event data augmentation with relevance propagation guidance (arXiv:2403.09274 [cs])
[50] Sabater A, Montesano L and Murillo A C 2022 Event transformer. a sparse-aware solution for efficient event data processing (arXiv:2204.03355 [cs])
[51] Furber S B, Lester D R, Plana L A, Garside J D, Painkras E, Temple S and Brown A D 2013 *IEEE Trans. Comput.* **62** 2454–67
[52] Kingma D P and Ba J 2017 Adam: a method for stochastic optimization (arXiv:1412.6980 [cs])
[53] Ansel J *et al* 2024 PyTorch 2: faster machine learning through dynamic python bytecode transformation and graph compilation *Proc. 29th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems*, Volume 2 (ASPLOS ' 24 vol 2) (Association for Computing Machinery) pp 929–47
[54] Merolla P A *et al* 2014 *Science* **345** 668–73
[55] Li Y, Geller T, Kim Y and Panda P 2023 SEENN: towards temporal spiking early-exit neural networks (arXiv:2304.01230 [cs])
[56] Liu C, Qi X, Lam E Y and Wong N 2022 *IEEE Access* **10** 55638–49
[57] Kinavuidi C 2025 Implementation for "Hyperdimensional decoding of spiking neural networks" *Zenodo* (https://doi.org/10.5281/zenodo.17522304)